

作业答案-类属和模板

1. 分别使用通用指针类型参数、函数模板实现类属的选择排序函数。

答:

(1) 用指针实现

```
void sel_sort(void *base, //需排序的数据首地址
              unsigned int n, //数据元素的个数
              unsigned int element_size, //数据元素的尺寸
              int (*cmp)(const void *, const void *)
              //比较两个数组元素大小的函数指针
              )
{ for ( ; n>1; n--) //基于数的个数n进行循环, 每次减少一个数
  { int i_max=0; //i_max用于保存最大元素的下标, 首先假设第 0 个元素最大
    for (int i=1; i<n; i++) //循环找前n个数中的最大元素
      if (cmp((char *)base+i*element_size, (char
*)base+i_max*element_size) > 0)
        i_max = i; //修改i_max的值, 使其一直为最大元素的下标
    //交换第i_max 个和第n-1 个元素
    char *p1=(char *)base+i_max*element_size,
        *p2=(char *)base+(n-1)*element_size;
    for (int k=0; k<element_size; k++)
    { char temp=p1[k];
      p1[k] = p2[k];
      p2[k] = temp;
    }
  }
}
```

(2) 用模板实现

```
template <class T> //T为元素类型
void sel_sort(T x[], int n)
{ for ( ; n>1; n--) //基于数的个数进行循环, 每次减少一个数
  { int i_max=0; //i_max用于保存最大元素的下标, 首先假设第 0 个元素最大
    for (int i=1; i<n; i++) //循环找前n个数中的最大元素
      if (x[i] > x[i_max]) i_max = i; //修改i_max的值, 使其一直为最大元素的下
标
    //交换x[i_max]和x[n-1]的值
    T temp = x[i_max];
    x[i_max] = x[n-1];
    x[n-1] = temp;
  }
}
```

2. 用类模板定义一个类属的队列类。

答:

```
template <class T, int size>
class Queue
{   T buffer[size+1]; //size个位置用于存放元素, 剩下的一个用于队列管理 (空/满)
    int front, rear;
public:
    Queue() { front = rear = 0; }
    bool enqueue(const T &x)
```

```

    { if ((rear+1)%(size+1) == front) //队列满
        return false;
        buffer[rear] = x;
        rear = (rear+1)%(size+1);
        return true;
    }
    bool deque(T &x)
    { if (front == rear) //队列空
        return false;
        x = buffer[front];
        front = (front+1)%(size+1);
        return true;
    }
};

```

3. 用 STL 的容器 vector 和算法 partition 实现快速排序功能。

答:

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

class Compare
{   int value;
public:
    Compare(int i) { value = i; }
    bool operator() (int i)
    { return i <= value;
    }
};

void qsort(vector<int>::iterator it1,vector<int>::iterator it2)
{ if (it2-it1 < 2) return; //元素个数小于 2, 返回
    vector<int>::iterator it=partition(it1,it2,Compare(*it1)); //取第一个元素
    作为分隔元素, it-1 为分隔位置
    //交换分隔元素与分隔位置上元素的位置
    int temp=*(it-1);
    *(it-1) = *it1;
    *it1 = temp;

    qsort(it1,it-1); //对分隔位置之前的元素排序
    qsort(it,it2); //对分隔位置之后的元素排序
}

int main()
{   vector<int> v;
    v.push_back(5);
    v.push_back(10);
    v.push_back(8);
    v.push_back(3);
    v.push_back(6);
    v.push_back(7);
    v.push_back(4);
    v.push_back(1);
    v.push_back(2);
    v.push_back(9);
}

```

```

qsort(v.begin(),v.end());

//输出排序后容器v的元素
vector<int>::iterator it1=v.begin();
vector<int>::iterator it2=v.end();
while (it1 != it2)
{ cout << *it1 << ' ';
  ++it1;
}
cout << endl;

return 0;
}

```

4. 对一个元素为 int 型的容器，用 STL 的算法 for_each 计算该容器中所有大于某个值的元素的和。

答：

```

#include <iostream>
#include <vector>
#include <algorithm>
using namespace std;

class MatchSum
{   int sum;
    int value;
public:
    MatchSum(int v)
    { sum=0;
      value = v;
    }
    void operator()(int i)
    { if (i > value) sum += i;
    }
    int get_sum() { return sum; }
};

int main()
{   vector<int> vec;
    vec.push_back(1);
    vec.push_back(2);
    vec.push_back(3);
    vec.push_back(4);
    vec.push_back(5);
    vec.push_back(6);
    vec.push_back(7);
    vec.push_back(8);
    vec.push_back(9);

    cout << for_each(vec.begin(),vec.end(),MatchSum(5)).get_sum() << endl;

    return 0;
}

```

5. 用 STL 的容器 vector 和算法 sort 实现以下功能：从键盘输入学生的成绩信息，每个信息包括：学号、姓名以及 8 门课的成绩。然后按照平均成绩由高到低的顺序输出学生的学号、姓名以及平均成绩。

答:

```
#include <iostream>
#include <vector>
#include <algorithm>
#include <string>
using namespace std;

class Student
{
    string id;
    string name;
    int scores[8]; //这里也可以用一个vector<int>来实现
public:
    void input_data()
    {
        cin >> id >> name;
        for (int i=0; i<8; i++)
            cin >> scores[i];
    }
    void display() const
    {
        cout << id << ',' << name << ',';
        for (int i=0; i<8; i++)
            cout << scores[i] << ',';
    }
    int average() const
    {
        for (int i=0, sum=0; i<8; i++)
            sum += scores[i];
        return sum/8;
    }
};

bool compare_average(Student &st1, Student &st2) //比较st1和st2的平均成绩
{
    return st1.average() > st2.average();
}

void display_student_info(Student &st) //显示st的信息
{
    st.display();
    cout << endl;
}

int main()
{
    vector<Student> students;
    Student st;
    int t;
    for (cin>>t; t!=-1; cin>>t) //循环输入每个学生的信息，输入每个学生信息前先输入
        一个是否结束的标记（-1表示结束）
    {
        st.input_data();
        students.push_back(st);
    }

    sort(students.begin(), students.end(), compare_average);

    for_each(students.begin(), students.end(), display_student_info);

    return 0;
}
```